Chương 5

TIÊU CHUẨN ĐÁNH GIÁ HỆ THỐNG SOC TRÊN FPGA

System-on-Chip Design

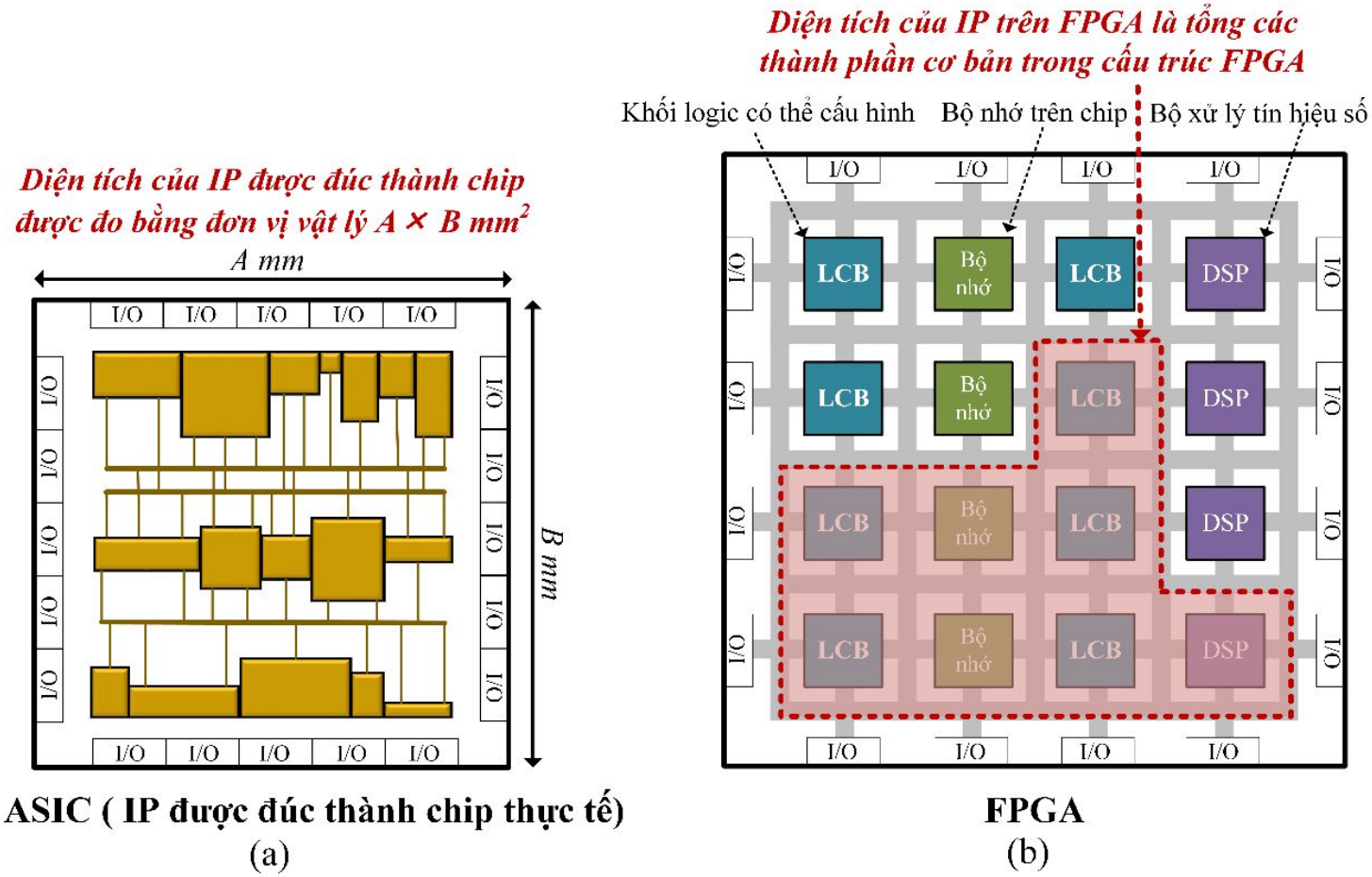
Mục lục

5.1. GIỚI THIỆU
5.2. ĐÁNH GIÁ TÀI NGUYÊN SỬ DỤNG
5.3. ĐÁNH GIÁ HIỆU NĂNG CỦA HỆ THỐNG
5.4. ĐÁNH GIÁ HIỆU QUẢ CỦA HỆ THỐNG
5.5. ĐỘ TIN CẬY
5.6. SỰ ĐÁNH ĐỔI TRONG HỆ THỐNG SOC
5.7. TÍCH HỢP HỆ THỐNG

GIỚI THIỆU

- Các tiêu chuẩn đánh giá hệ thống SoC trên FPGA là cần thiết để đảm bảo hiệu quả và độ tin cậy của hệ thống, bao gồm:
  - **Tài nguyên sử dụng trên FPGA**: Bao gồm diện tích (như LUTs, FFs, DSPs, BRAMs) và năng lượng, ảnh hưởng trực tiếp đến khả năng tích hợp và tối ưu của hệ thống, diện tích khi chuyển đổi thiết kế từ FPGA sang IP hoặc chip thực tế.
  - **Hiệu năng hệ thống**: phản ánh khả năng xử lý của SoC khi thực hiện các tác vụ có độ chính xác và tốc độ cao.
  - **Hiệu quả hệ thống**: gồm hai yếu tố là hiệu quả diện tích và hiệu quả năng lượng, cần phải tìm ra sự cân bằng hợp lý giữa hiệu năng, diện tích và năng lượng để đạt được một hệ thống tối ưu.
  - **Độ tin cậy**: Hệ thống SoC đáng tin cậy phải có khả năng chịu đựng các lỗi phát sinh từ phần cứng, phần mềm, và môi trường như nhiệt độ cao, nhiễu điện từ, hoặc suy hao điện áp, giúp duy trì hiệu năng của hệ thống mà còn giảm chi phí bảo trì và tăng tuổi thọ sản phẩm
  - **Tích hợp hệ thống:** giúp hệ thống dễ dàng thích ứng với yêu cầu mới hoặc tích hợp công nghệ trong tương lai mà không cần thay đổi toàn bộ kiến trúc

ĐÁNH GIÁ TÀI NGUYÊN SỬ DỤNG



Đánh giá diện tích của IP trên chip ASIC và FPGA: (a) Diện tích của IP trên chip ASIC được đo bằng các đơn vị vật lý; (b) Diện tích của IP trên FPGA là tổng diện tích của các thành phần cơ bản trong cấu trúc FPGA.

ĐÁNH GIÁ DIỆN TÍCH TRÊN ALTERA FPGA DÙNG PHẦN MỀM QUARTUS PRIME

Family	Cyclone IV E
Device	EP4CE115F29C7
Timing Models	Final
Total logic elements	2,727 / 114,480 (2 %)
Total registers	1496
Total pins	50 / 529 (9 %)
Total virtual pins	0
Total memory bits	1,231,492 / 3,981,312 (31 %)
Embedded Multiplier 9-bit elements	26 / 532 (5 %)
Total PLLs	0 / 4 (0 %)

Diện tích trên Altera FPGA trích xuất tài nguyên từ phần mềm Quartus Prime.

ĐÁNH GIÁ DIỆN TÍCH TRÊN XILINX FPGA DÙNG PHẦN MỀM VIVADO

Name	CLB LUTs (274080)	CLB Registers (548160)	CARRY8 (34260)	F7 Muxes (137040)	CLB (34260)	LUT as Logic (274080)	LUT as Memory (144000)	Block RAM Tile (912)	DSPs (2520)	GLOBAL CLOCK BUFFERs (404)	PS8 (1)
SoC_wrapper	31926	20637	300	16	5922	26990	4936	4.5	40	1	1
SoC_i (SoC)	31926	20637	300	16	5922	26990	4936	4.5	40	1	1
axi_smc (SoC_axi_s	7059	9463	16	0	1459	5404	1655	0	0	0	0
CGRA_0 (SoC_CGRA	24855	11111	284	16	4471	21575	3280	4.5	40	0	0
rst_ps8_0_99M (Sc	13	63	0	0	36	12	1	0	0	0	0
zynq_ultra_ps_e_0	0	0	0	0	0	0	0	0	0	1	1

Diện tích SoC trên ZCU102 FPGA trích xuất tài nguyên từ phần mềm Vivado.

QUY ĐỔI TÀI NGUYÊN DSP SANG LUT VÀ FF

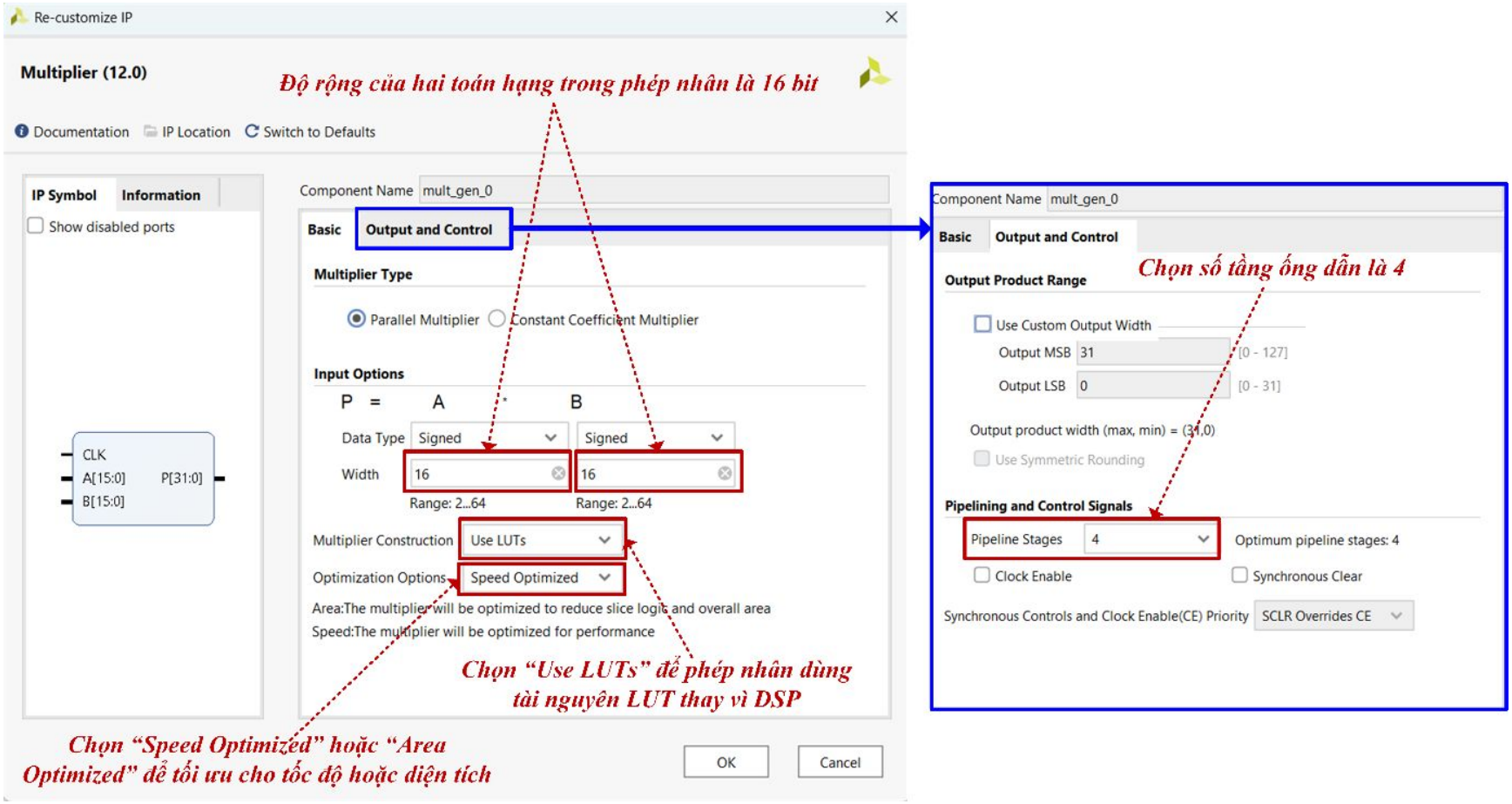
```
module multiplier (
    input wire clk,           // Xung clock
    input wire rst,           // Tín hiệu reset
    input wire [15:0] a,      // Đầu vào a, 16-bit
    input wire [15:0] b,      // Đầu vào b, 16-bit
    output reg [31:0] product // Đầu ra là kết quả phép nhân, 32-bit
);

    // Mô tả hành vi của module nhân
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            // Nếu có tín hiệu reset, đưa kết quả về 0
            product <= 32'b0;
        end else begin
            // Nếu không có reset, thực hiện phép nhân
            product <= a * b;
        end
    end

endmodule
```

Code Verilog mô tả phép nhân với độ rộng 16 bit cho hai toán hạng mà không sử dụng IP phép nhân có sẵn trong IP Catalog của Vivado.

QUY ĐỔI TÀI NGUYÊN DSP SANG LUT VÀ FF



Cấu hình phép nhân với độ rộng 16 bit cho hai toán hạng, chọn tối ưu tốc độ, và chọn tầng ống dẫn là 4, sử dụng LUT thay vì DSP trên phần mềm Vivado.

QUY ĐỔI TÀI NGUYÊN DSP SANG LUT VÀ FF

Name	CLB LUTs (274080)	CLB Registers (548160)	CARRY8 (34260)	Bonded IOB (328)	GLOBAL CLOCK BUFFERs (404)
mult_gen_0	282	301	45	65	1

Diện tích phép nhân với độ rộng 16 bit cho hai toán hạng, tối ưu hóa tốc độ, và 4 tầng đường ống trên ZCU102 FPGA.

Name	CLB LUTs (274080)	CLB Registers (548160)	CARRY8 (34260)	Bonded IOB (328)	GLOBAL CLOCK BUFFERs (404)
mult_gen_0	1466	1587	92	129	1

Diện tích phép nhân với độ rộng 32 bit cho hai toán hạng, tối ưu hóa diện tích, và 8 tầng đường ống trên ZCU102 FPGA.

QUY ĐỔI TÀI NGUYÊN DSP SANG LUT VÀ FF

Name 1	CLB LUTs (274080)	CLB Registers (548160)	CARRY8 (34260)	Bonded IOB (328)	GLOBAL CLOCK BUFFERs (404)
> N mult_gen_0	280	32	45	65	1

Diện tích phép nhân với độ rộng 16 bit cho hai toán hạng và 1 tầng ống dẫn của IP tương đương với mã Verilog mô tả trong Slide 10

QUY ĐỔI BRAM SANG LUT VÀ FF

- BRAM được sử dụng để lưu trữ dữ liệu hoặc làm bộ nhớ cache, nhưng có thể thay thế bằng LUT và FF.
- BRAM thường chia làm 2 loại đơn vị: BRAM 18 Kb và BRAM 36 Kb. Cụ thể, một BRAM (18 Kb) tương đương với 0.5 BRAM và BRAM (36 Kb) tương đương với 1 BRAM trên các Xilinx FPGA.
- Một BRAM (18 Kb) thường tương đương xấp xỉ 32-512 FF và xấp xỉ 800 LUT.
- Một BRAM (36 Kb) thường tương đương xấp xỉ 32-512 FF và xấp xỉ 1600 LUT.
- Khi thực hiện quy đổi, tổng tài nguyên của một thiết kế được tính bằng cách cộng tổng các tài nguyên đã quy đổi về LUT và FF:

$$Tổng\ tài\ nguyên = FF_{thực\ tế} + LUT_{thực\ tế} + (DSP\ quy\ đổi) + (BRAM\ quy\ đổi)$$

QUY ĐỔI BRAM SANG LUT VÀ FF

- **Ví dụ**: Thiết kế thứ nhất sử dụng 10 DSP (phép nhân với độ rộng 16 bit cho hai toán hạng và 1 tầng ống dẫn), 2 BRAM (36Kb), 3000 LUT và 3500 FF. Và thiết kế thứ hai sử dụng 15 DSP (phép nhân với độ rộng 16 bit cho hai toán hạng và 1 tầng ống dẫn), 6 BRAM (36Kb), 1500 LUT và 2000 FF. Thực hiện so sánh hiệu quả của hai thiết kế trên. Biết rằng 1 DSP tương đương với 280 LUT và 32 FF, và 1 BRAM (36Kb) tương đương 1,600 LUT và 32 FF.

- **Thiết kế 1**
  - DSP block (10): LUT: 10×280 = 2,800; FF: 10×32 = 320
  - Block RAM (2): LUT: 2×1,600 = 3,200; FF: 2×32 = 64
  - FF thực tế: 5,000.
  - LUT thực tế: 3,000.
  - Tổng tài nguyên:
  - LUT: 3,000 + 2,800 + 3,200 = 9,000; FF: 3,500 + 320 + 64 = 3,884
- **Thiết kế 2**
  - DSP block (15): LUT: 15×280 = 4,200; FF: 15×32 = 480
  - Block RAM (6): LUT: 6×1,600 = 9,600; FF: 6×32 = 192
  - FF thực tế: 2,000.
  - LUT thực tế: 1,500.
  - Tổng tài nguyên:
  - LUT: 1,500 + 4,200 + 9,600 = 15,300; FF: 2,000 + 480 + 192 = 2,672

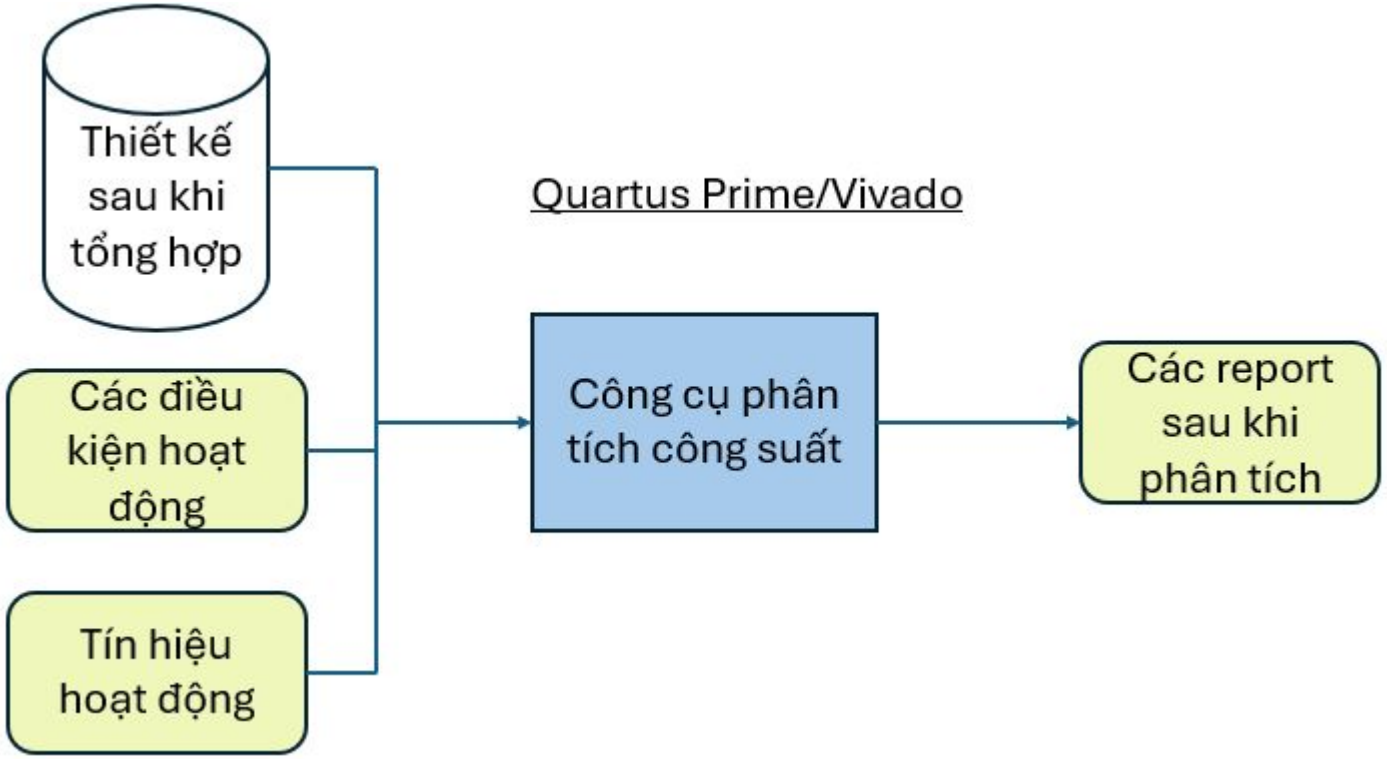
QUY ĐỔI BRAM SANG LUT VÀ FF

So sánh tổng tài nguyên giữa hai thiết kế

Tài nguyên	Thiết kế 1	Thiết kế 2
LUT	9,000	15,300
FF	3,884	2,672

- Mặc dù thiết kế 1 ban đầu có giá trị LUT và FF lớn hơn thiết kế 2, nhưng tổng giá trị LUT và FF của thiết kế 1 lại nhỏ hơn, với 9,000 LUT và 5,384 FF so với 15,300 LUT và 2,672 FF của thiết kế 2.
- LUT thường là tiêu chí chính để so sánh các thiết kế vì nó ảnh hưởng trực tiếp đến khả năng thực hiện phép toán và xử lý dữ liệu.
- FF ít được xem xét vì nó chỉ liên quan đến lưu trữ dữ liệu tạm thời và không ảnh hưởng nhiều đến hiệu suất tổng thể.
- Kết luận, thiết kế 1 có diện tích nhỏ hơn và hiệu quả hơn thiết kế 2, làm cho nó phù hợp hơn cho các ứng dụng đòi hỏi tiết kiệm tài nguyên và tối ưu hóa hiệu suất.

ĐÁNH GIÁ NĂNG LƯỢNG



Quy trình phân tích công suất cho hệ thống SoC trên FPGA.

ĐÁNH GIÁ NĂNG LƯỢNG

- Công suất tiêu thụ của FPGA thường được chia thành hai loại chính:
  - Công suất tĩnh (Static Power): Là công suất tiêu thụ ngay cả khi FPGA không thực hiện bất kỳ hoạt động nào, chủ yếu do dòng rò và điện áp cung cấp. Công suất tĩnh phụ thuộc vào công nghệ chế tạo chip, nhiệt độ hoạt động, và điện áp cung cấp.
  - Công suất động (Dynamic Power): Là công suất tiêu thụ khi các khối logic và mạch giao tiếp hoạt động.

Đánh giá năng lượng trên Altera FPGA dùng phần mềm Quartus Prime

PowerPlay Power Analyzer Summary	
PowerPlay Power Analyzer Status	Successful - Wed Jan 22 20:18:52 2025
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	fifidemo
Top-level Entity Name	fifidemo
Family	Cyclone II
Device	EP2C5T144C6
Power Models	Final
Total Thermal Power Dissipation	31.48 mW
Core Dynamic Thermal Power Dissipation	0.00 mW
Core Static Thermal Power Dissipation	18.01 mW
I/O Thermal Power Dissipation	13.47 mW
Power Estimation Confidence	Low: user provided insufficient toggle rate data

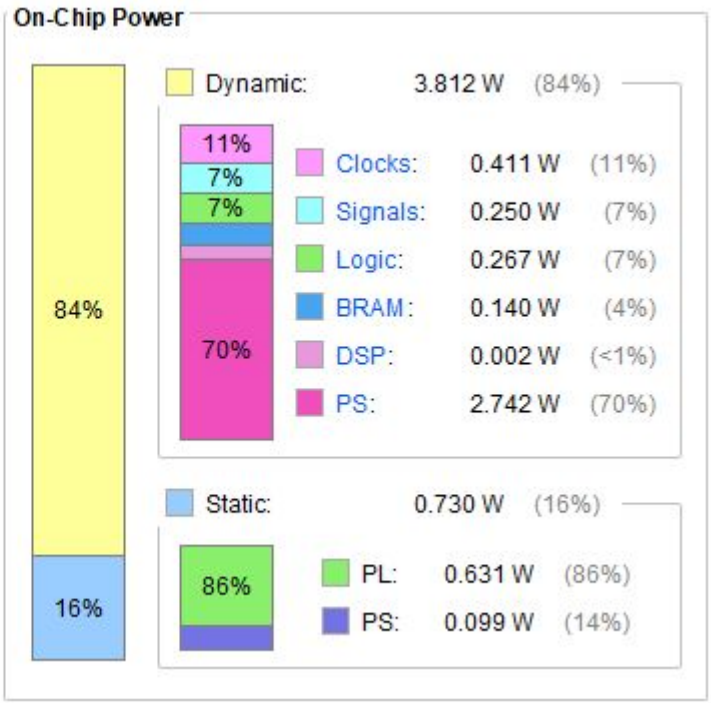
Năng lượng trên Altera FPGA trích xuất tài nguyên từ phần mềm Quartus Prime..

Đánh giá năng lượng trên Xilinx FPGA dùng phần mềm Vivado

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**4.542 W**
Design Power Budget:	**Not Specified**
Power Budget Margin:	**N/A**
Junction Temperature:	**29,4°C**
Thermal Margin:	70,6°C (70,4 W)
Effective ϑJA:	1,0°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Medium

Launch Power Constraint Advisor to find and fix invalid switching activity

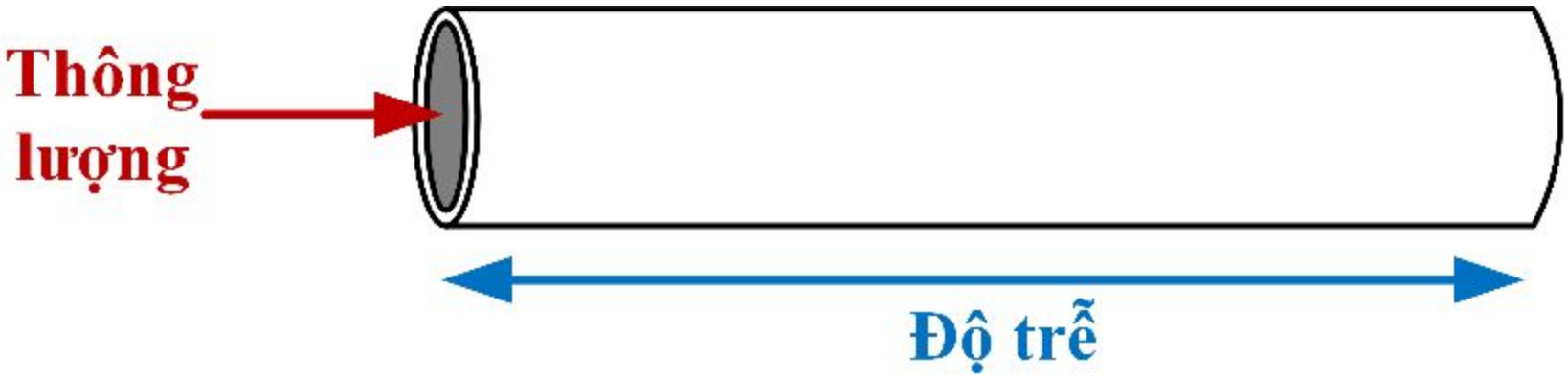


Ví dụ trích xuất thông tin công suất tiêu thụ từ công cụ phân tích công xuất được tích hợp trong Vivado Xilinx

Phương pháp giảm năng lượng tiêu thụ

- Một số giải pháp giúp giảm công suất tiêu thụ cho hệ thống SoC trên FPGA thông qua việc tối ưu thiết kế:
  - Tối ưu hóa thiết kế logic: Loại bỏ các logic không sử dụng (Unused Logic Removal). Ưu tiên thiết kế đồng bộ hóa hệ thống thay vì bất đồng bộ hóa.
  - Tối ưu hóa đường dữ liệu (Data Path Optimization): Sử dụng khối DSP thay cho LUT/FF để thực hiện các phép toán phức tạp nhằm giảm số lần chuyển mạch.
  - Tối ưu hóa bộ nhớ: Tận dụng bộ nhớ cục bộ như BRAM hoặc URAM trên FPGA để giảm truy cập bộ nhớ ngoài. Sử dụng các kỹ thuật quản lý bộ nhớ hiệu quả để giảm độ trễ và công suất.
  - Tối ưu hóa giao tiếp I/O: Giảm hoạt động I/O không cần thiết hoặc sử dụng chế độ I/O tiết kiệm năng lượng.
  - Thiết kế kiến trúc song song hiệu quả: Sử dụng các khối xử lý song song tối ưu để tăng throughput mà không tăng tần số xung nhịp. Chia nhỏ dữ liệu và sử dụng các đơn vị xử lý cục bộ để giảm truy cập tài nguyên chung.
  - Tối ưu hóa hoạt động của FPGA: Sử dụng IP tối ưu năng lượng: Chọn các IP cứng thay cho IP mềm khi có thể, vì IP cứng thường tối ưu hóa năng lượng tốt hơn.

Đánh giá hiệu năng của hệ thống



Hình ảnh trực quan giải thích khái niệm về độ trễ và thông lượng.

Đánh giá hiệu năng của hệ thống

- Để tính được độ trễ của hệ thống, ta cần thực hiện các bước sau:
  - Xác định chu kỳ dài nhất của hệ thống: Chu kỳ dài nhất phản ánh thời gian cần thiết để hoàn thành một chu kỳ xử lý trong hệ thống.
  - Xác định tần số hoạt động tối đa của hệ thống: Tần số hoạt động tối đa xác định tốc độ mà các tác vụ trong hệ thống có thể được thực hiện.
  - Đánh giá tầm quan trọng của chu kỳ dài nhất và tần số hoạt động tối đa:
    - Chu kỳ dài nhất và tần số hoạt động tối đa là hai yếu tố tiên quyết quan trọng để tính toán độ trễ và thông lượng.
    - Độ trễ và thông lượng trực tiếp phụ thuộc vào thời gian cần thiết để xử lý và truyền tải dữ liệu qua các thành phần trong hệ thống.
  - Hiểu rõ tần số hoạt động của hệ thống: Trước khi đi vào các phương pháp đánh giá độ trễ và thông lượng, ta cần hiểu rõ tần số hoạt động của hệ thống để đưa ra những ước lượng chính xác về hiệu năng của hệ thống trong quá trình xử lý dữ liệu.

Ước tính chu kỳ dài nhất và tần số hoạt động tối đa

- Các thông số quan trọng cần được kiểm tra như Slack âm nhất (WNS - Worst Negative Slack) và tổng số Slack âm (TNS - Total Negative Slack):
  - WNS: là giá trị Slack nhỏ nhất (âm nhất) trong tất cả các đường tín hiệu trong thiết kế. Slack là chênh lệch giữa thời gian yêu cầu (Required Time) và thời gian thực hiện thực tế (Arrival Time) của một tín hiệu. Nếu WNS≥0: Tất cả các đường tín hiệu đều đáp ứng yêu cầu thời gian => thiết kế hợp lệ. Nếu WNS<0: Có ít nhất một đường tín hiệu không đáp ứng yêu cầu thời gian, gây vi phạm thời gian (timing violation).

Công thức: **Slack = Thời gian yêu cầu - Thời gian thực tế**

Với:

Thời gian yêu cầu : Thời gian yêu cầu mà tín hiệu phải đến đích để đáp ứng chu kỳ xung nhịp.

Thời gian thực tế: Thời gian thực tế mà tín hiệu đến đích.

Ước tính chu kỳ dài nhất và tần số hoạt động tối đa

- Các thông số quan trọng cần được kiểm tra như Slack âm nhất (WNS - Worst Negative Slack) và tổng số Slack âm (TNS - Total Negative Slack):
  - **TNS**: là tổng tất cả các giá trị Slack âm trong thiết kế. Nó đại diện cho tổng mức độ vi phạm thời gian trên toàn bộ các đường tín hiệu. TNS = 0: Thiết kế không có vi phạm thời gian, tất cả các đường tín hiệu đều đáp ứng yêu cầu. TNS > 0: Không thể xảy ra vì chỉ tính Slack âm. TNS < 0: Giá trị âm biểu thị tổng mức độ vi phạm thời gian trong thiết kế. TNS càng nhỏ, thiết kế càng có nhiều lỗi thời gian nghiêm trọng và cần tối ưu hóa.

Công thức: **TNS= slack với các đường tín hiệu có Slack < 0**

  - **Đường dài nhất:** là đường tín hiệu dài nhất (critical path) từ đầu vào đến đầu ra, với thời gian trễ lớn nhất. Critical Path Delay: thời gian trễ tổng cộng của tín hiệu khi di chuyển dọc theo đường critical path, từ đầu vào đến đầu ra. Công thức tính độ trễ đường dài nhất (Critical Path Delay) $T_{\text{dài nhất}}$ (T_{critical}) khi ta biết tần

$$T_{\text{dài nhất}} = \text{Chu kỳ xung đồng hồ}_{\text{Khởi tạo}} - \text{WNS} \qquad (5.3)$$

$$\text{Ta có :} \quad F_{MAX} = \frac{1}{T_{\text{dài nhất}}} \qquad (5.4)$$

Ví dụ: Một hệ thống SoC được khởi tạo giá trị tần số hoạt động ban đầu 100MHz. Sau khi tổng hợp, từ thông tin thời gian ta trích xuất được thông số WNS bằng 0.25 ns. Tìm tần số hoạt động tối đa của hệ thống.

$$T_{\text{dài nhất}} = \frac{1}{F_{\text{Khởi tạo}}} - WNS = 10\ ns - 0.25\ ns = 9.75\ ns$$

Suy ra $$F_{MAX} = \frac{1}{T_{\text{dài nhất}}} = \frac{1}{9.75\ ns} = 102.56\ MHz$$

Đánh giá độ trễ

- Trong hệ thống SoC trên FPGA, chúng ta thường dùng chỉ số độ đễ (latency) để đánh giá hệ thống. Độ trễ thường được định nghĩa là khoảng thời gian từ khi một tác vụ hoặc dữ liệu được bắt đầu xử lý ở đầu vào đến khi kết quả hoàn tất ở đầu ra.

Công thức chung:

$$\text{Độ } trễ\ (tổng) = \sum_{i=1}^{N} \text{độ } trễ_{\ thành\ phần\ i} + \text{độ } trễ_{giao\ tiếp}$$

- Trong đó:
  - N: Số lượng thành phần trong đường dữ liệu (datapath)
  - Độ trễ thành phần i: Độ trễ của từng thành phần, bao gồm CPU, bộ nhớ, bus kết nối, IP
  - Độ trễ giao tiếp: Độ trễ phát sinh từ việc truyền dữ liệu giữa các thành phần qua bus, giao tiếp nội, hoặc các cơ chế giao tiếp khác.

Đánh giá độ trễ

Đối với IP CPU

$$\text{Độ } trễ_{CPU} = \frac{Số\ chu\ kỳ\ xử\ lý}{Tần\ số\ hoạt\ động\ của\ CPU} \quad (5.6)$$

Trong đó số chu kỳ xử lý phụ thuộc vào đặc trưng cấu tạo của CPU, tập lệnh CPU, và cấu trúc toán hoạn của CPU.

Đối với IP bộ nhớ

$$\text{Độ } trễ_{bộ\ nhớ} = Thời\ gian\ truy\ cập\ cơ\ bản\ (Access\ Time) + \text{Độ } trễ\ giao\ tiếp\ (Transfer\ Latency) \quad (5.7)$$

Trong đó:

- Thời gian truy cập cơ bản: Là thời gian từ lúc bắt đầu yêu cầu đọc/ghi (read/write) đến khi dữ liệu sẵn sàng để truyền.
- Độ trễ giao tiếp: Thời gian cần thiết để truyền dữ liệu từ bộ nhớ đến thiết bị yêu cầu qua bus hoặc giao tiếp.

Đánh giá độ trễ

Đối với IP bus hoặc kết nối nội (interconnect)

$$\text{Độ trễ}_{bus} = \frac{\text{Số chu kỳ giao tiếp}}{\text{Tần số bus}} \quad (5.12)$$

Đối với IP hoặc khối logic tự thiết kế:

Trường hợp không có pipeline:

$$\text{Độ trễ}_{\text{IP không pipeline}} = \frac{\text{Số chu kỳ cần thiết cho IP hoạt động}}{\text{Tần số hoạt động của IP}} \quad (5.13)$$

Trường hợp có pipeline:

$$\text{Độ trễ}_{\text{IP có pipeline}} = \frac{\text{Độ trễ}_{\text{IP không pipeline}}}{\text{Số tần pipeline}} \quad (5.14)$$

Trường hợp IP hoạt động lặp lại:

$$\text{Độ trễ}_{IP} = \frac{\text{Số chu kỳ mỗi tác vụ} \times \text{số lần lặp}}{\text{Tần số hoạt động của IP}} \quad (5.15)$$

Đánh giá độ trễ

- *Ví dụ: Giả sử một hệ thống SoC gồm CPU ARM Cortex có tần số hoạt động: 1 GHz, thời gian xử lý trung bình một tác vụ: 50 chu kỳ xung nhịp CPU. Một bộ nhớ trong (on chip memory) có độ trễ truy cập là 2 chu kỳ CPU. Bus AXI có độ trễ truyền nhận (mỗi chiều): 15 chu kỳ CPU. Một IP hoạt động ở tần số 200 MHz, biết tổng số chu kỳ tính toán của IP là 100 chu kỳ. Tính độ trễ của toàn bộ hệ thống SoC trong trường hợp IP không có pipeline và có pipeline. Giả sử khi có pipeline IP sẽ được thiết kế với 5 tầng pipeline.*

Đánh giá độ trễ

Độ trễ xử lý của CPU:

$$\text{Độ trễ } (CPU) = \frac{50 \text{ chu kỳ}}{\text{Tần số } CPU} = \frac{50}{1 \times 10^9} = 50\ ns$$

Độ trễ truy cập bộ nhớ:

$$\text{Độ trễ } (\text{bộ nhớ}) = \frac{2 \text{ chu kỳ}}{\text{Tần số } CPU} = \frac{2}{1 \times 10^9} = 2\ ns$$

Độ trễ đường truyền bus:

$$\text{Độ trễ } (bus) = 2 \times \frac{15 \text{ chu kỳ}}{\text{Tần số } CPU} = 2 \times \frac{15}{1 \times 10^9} = 30\ ns$$

Độ trễ của IP:

Trường hợp 1: không pipeline

$$\text{Độ trễ } (IP) = \frac{100 \text{ chu kỳ}}{\text{Tần số } IP} = \frac{100}{200 \times 10^6} = 500\ ns$$

Trường hợp 2: có pipeline

$$\text{Độ trễ } (IP) = \frac{500}{5} = 100\ ns$$

Độ trễ toàn hệ thống SoC được tính như sau:

Trường hợp 1: không pipeline

$$\text{Độ trễ } (\text{toàn bộ})$$
$$= \text{độ trễ } (CPU) + \text{độ trễ } (\text{bộ nhớ}) + \text{độ trễ } (bus)$$
$$+ \text{độ trễ } (IP \text{ không pipeline}) + \text{độ trễ } (bus)$$
$$= 50 + 2 + 30 + 500 + 30 = 612\ ns$$

Trường hợp 2: có pipeline

$$\text{Độ trễ } (\text{toàn bộ}) = \text{độ trễ } (CPU) + \text{độ trễ } (\text{bộ nhớ}) + \text{độ trễ } (bus) +$$
$$\text{độ trễ } (IP \text{ có pipeline}) + \text{độ trễ } (bus)$$
$$= 50 + 2 + 30 + 100 + 30 = 212\ ns$$

Đánh giá thông lượng

- Thông lượng (throughput) trong hệ thống SoC FPGA là lượng dữ liệu mà hệ thống có thể xử lý trong một đơn vị thời gian

Thông lượng = Số lượng dữ liệu xử lý / Thời gian thực hiện

Hoặc

Thông lượng bps=tần số hoạt động × số bit dữ liệu xử lý trong một chu kỳ

- Yếu tố ảnh hưởng đến thông lượng
  - **Tần số hoạt động**: Tăng tần số hoạt động cho phép hệ thống thực hiện nhiều chu kỳ xử lý hơn trong cùng một khoảng thời gian, từ đó cải thiện thông lượng.
  - **Độ rộng dữ liệu (Data width)**: Việc tăng độ rộng dữ liệu, tức là số bit được xử lý mỗi chu kỳ, giúp hệ thống xử lý lượng dữ liệu lớn hơn trong một khoảng thời gian, nâng cao hiệu năng thông lượng.
  - **Độ trễ**: Giảm độ trễ trong quá trình xử lý, đặc biệt trong các hệ thống xử lý liên tục, có thể trực tiếp cải thiện thông lượng bằng cách giảm thời gian chờ giữa các lệnh.
  - **Kiến trúc song song**: Triển khai nhiều đường dữ liệu song song trên FPGA cho phép xử lý đồng thời nhiều luồng dữ liệu, mang lại sự cải thiện đáng kể về thông lượng của hệ thống.

Đánh giá hiệu quả của hệ thống

1. **Đánh giá hiệu quả diện tích:**

Tích diện tích trễ (ADP) :

$$ADP = \text{Diện tích} \times \text{Độ trễ}$$

- Trong đó:

✔ Diện tích: Là tổng số LUT sử dụng trong thiết kế.

✔ Độ trễ: Là thời gian cần thiết để hoàn thành một tác vụ hoặc truyền tải dữ liệu trong hệ thống.

ADP càng nhỏ thì hệ thống càng tốt, phản ánh được hệ thống vừa có diện tích nhỏ và độ trễ nhỏ, hoặc hệ thống có diện tích rất nhỏ và độ trễ chấp nhận được, hoặc hệ thống có diện tích chấp nhận được và độ trễ rất thấp.

Đánh giá hiệu quả của hệ thống

1. **Đánh giá hiệu quả diện tích:**

Thông lượng trên mỗi đơn vị diện tích:

Thông lượng trên mỗi đơn vị diện tích= Thông lượng Diện tích

- Trong đó:

✔ Diện tích: Là tổng số LUT sử dụng trong thiết kế.

✔ Thông lượng: Là lượng dữ liệu mà hệ thống có thể xử lý trong một khoảng thời gian nhất định (thường là bit/giây hoặc phép toán/giây).

Chỉ số thông lượng trên mỗi đơn vị diện tích càng lớn càng tốt, phản ánh khả năng hệ thống có thể xử lý dữ liệu hiệu quả mà không cần sử dụng quá nhiều tài nguyên phần cứng.

Đánh giá hiệu quả của hệ thống

2. Đánh giá hiệu quả năng lượng:

Tích năng lượng độ trễ (PDP)

$$PDP = Năng\ lượng \times Độ\ trễ$$

- Trong đó:

✔ Năng lượng: Là năng lượng tiêu thụ của thiết kế, tính bằng Watt hoặc mili Watt.

✔ Độ trễ: Là thời gian cần thiết để hoàn thành một tác vụ hoặc truyền tải dữ liệu trong hệ thống.

PDP càng nhỏ thì hệ thống càng tốt, phản ánh được hệ thống vừa có năng lượng tiêu thụ thấp và độ trễ nhỏ, hoặc hệ thống có năng lượng tiêu thụ rất thấp và độ trễ chấp nhận được, hoặc hệ thống có năng lượng tiêu thụ chấp nhận được và độ trễ rất thấp.

Đánh giá hiệu quả của hệ thống

2. Đánh giá hiệu quả năng lượng:

Thông lượng trên mỗi đơn vị năng lượng

Thông lượng trên mỗi đơn vị năng lượng= Thông lượng Năng lượng

- Trong đó:

✔ Năng lượng: Là năng lượng tiêu thụ của thiết kế, tính bằng Watt hoặc mili Watt.

✔ Thông lượng: Là lượng dữ liệu mà hệ thống có thể xử lý trong một khoảng thời gian nhất định (thường là bit/giây hoặc phép toán/giây).

Chỉ số thông lượng trên mỗi đơn vị năng lượng càng lớn càng tốt, phản ánh khả năng hệ thống có thể xử lý dữ liệu hiệu quả trong khi tiêu thụ ít năng lượng.

Ví dụ: Giả sử chúng ta có hai thiết kế hệ thống SoC. Thiết kế 1 có thông lượng là 5 Gbps và diện tích là 5000 LUT, trong khi Thiết kế 2 có thông lượng là 2 Gbps và diện tích là 1000 LUT. Xác định thiết kế nào hiệu quả hơn.

Đầu tiên, chúng ta tính thông lượng trên mỗi đơn vị diện tích cho mỗi thiết kế.

Thiết kế 1:

$$Thông\ lượng\ trên\ mỗi\ đơn\ vị\ \ diện\ tích\ 1 = \frac{5\ Gbps}{5000\ LUT} = \frac{5000\ Mbps}{5000\ LUT}$$

$$= 1\ Mbps/LUT$$

Thiết kế 2:

$$Thông\ lượng\ trên\ mỗi\ đơn\ vị\ \ diện\ tích\ 2 = \frac{2\ Gbps}{1000\ LUT} = \frac{2000\ Mbps}{1000\ LUT}$$

$$= 2\ Mbps/LUT$$

Ví dụ: Giả sử một hệ thống SoC có CPU, bộ nhớ và một IP tự thiết kế thực hiện bài toán nhân hai ma trận có kích thước $M \times K$ *và* $K \times N$. *Mỗi phần tử là một số thực (float 32 bit). Tần số hoạt động của FPGA là 200 MHz, tần số hoạt động của IP tự thiết kế là 500 MHz, độ trễ truy cập bộ nhớ 100 ns, độ trễ bus là 50 ns, các độ trễ khác xem như bỏ qua. Biết mỗi phép tính nhân cộng dồn (MAC – Multiply-Accumulate) tốn 5 ns và giả sử mỗi phép tính cần tải dữ liệu từ bộ nhớ một lần. Tính toán thông lượng của hệ thống SoC này trong trường hợp IP thiết kế không xử ý song song và có xử lý song song.*

Độ tin cậy

- Độ tin cậy trong thiết kế vi mạch SoC đề cập đến khả năng của một vi mạch để hoạt động đúng theo các thông số kỹ thuật đã được định trước, trong suốt một khoảng thời gian nhất định và dưới những điều kiện vận hành cụ thể.
- Bao gồm:
  - Khả năng chống lại lỗi hệ thống thông qua việc phát hiện và sửa lỗi
  - Duy trì hiệu năng
  - Độ bền và tuổi thọ của sản phẩm trong điều kiện sử dụng thực tế.
- Các yếu tố ảnh hưởng:
  - Các mạch hoạt động ở tốc độ cao thường được thiết kế nhỏ gọn hơn => dễ bị ảnh hưởng bởi bức xạ và các tác động môi trường khác, từ đó làm giảm độ tin cậy.
- FPGA nhờ tính năng tái cấu hình, đã nâng cao đáng kể độ tin cậy, đặc biệt trong các ứng dụng yêu cầu hoạt động ổn định và liên tục.
- Khi một phần mạch trên FPGA gặp sự cố, chẳng hạn do quá nhiệt hoặc hư hỏng vật lý, FPGA có thể tái cấu hình để "bỏ qua" phần mạch bị lỗi, đồng thời định tuyến lại các chức năng qua các khối logic còn hoạt động

=> FPGA không chỉ giảm thiểu rủi ro hỏng hóc hệ thống mà còn giảm chi phí thay thế phần cứng.

Lỗi và sửa lỗi

- **Lỗi tín hiệu**: Đây là lỗi do giá trị tín hiệu không chính xác, có thể xảy ra do nhiễu hoặc các vấn đề liên quan đến thiết kế đường truyền tín hiệu.
  - Ví dụ: Nếu một tín hiệu xung clock bị méo hoặc không đáp ứng yêu cầu thời gian, hệ thống có thể hoạt động không đúng. Để khắc phục, kỹ sư thường sử dụng các kỹ thuật như lọc tín hiệu, tối ưu hóa thiết kế mạch hoặc thêm các mạch kiểm tra lỗi (Error Detection Circuit).
- **Lỗi logic**: Lỗi logic thường biểu hiện dưới dạng kết quả đầu ra không đúng so với mong đợi.
  - Ví dụ: Trong một thiết kế FPGA thực hiện phép cộng, nếu kết quả phép toán luôn sai khi một số đầu vào cụ thể được đưa vào, thì đó là lỗi logic. Nguyên nhân có thể là do sai sót trong quá trình viết mã HDL (Hardware Description Language). Sửa lỗi này yêu cầu kiểm tra kỹ lưỡng logic thiết kế thông qua mô phỏng hoặc phân tích trạng thái.

Lỗi và sửa lỗi

Tốc độ phát sinh lỗi θ theo thời gian trung bình T với 1 xác suất lỗi P(t) được cho bởi công thức:

$$\theta = 1T$$

Và một xác xuất lỗi theo phân bố Poison được cho bởi công thức:

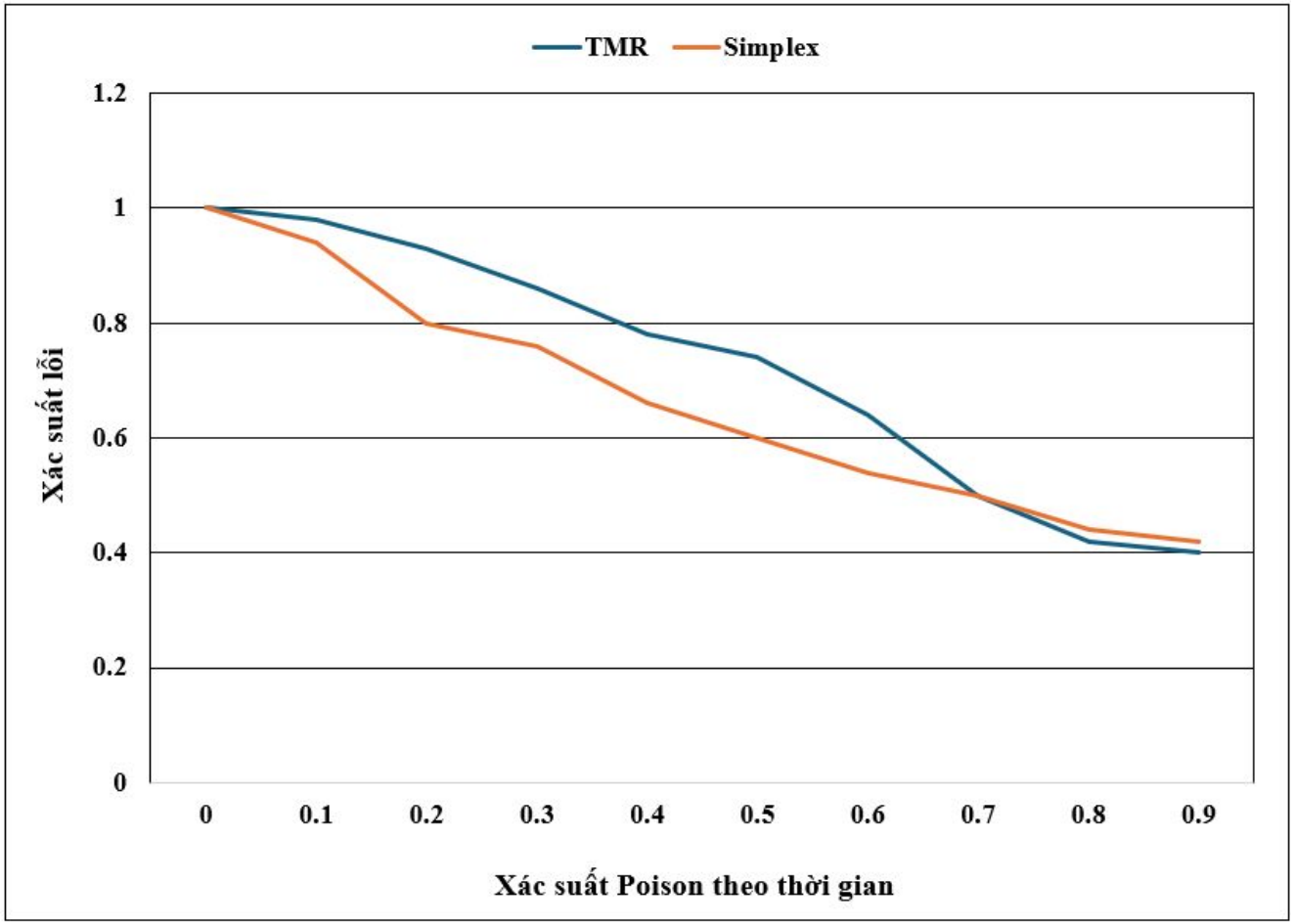
$$Pt = e\text{-}tT = e\text{-}t\theta$$

=> Khi đó để cải thiện độ tin cậy, chúng ta phải làm giảm giá trị P(t) xuống.

Kỹ thuật cải tiến độ tin cậy

- TMR (triple modular redundancy) là một kỹ thuật cải tiến độ tin cậy cao được sử dụng trong thiết kế hệ thống điện tử, đặc biệt là trong những ứng dụng đòi hỏi mức độ an toàn và độ tin cậy rất cao như hệ thống điều khiển hàng không và y tế.
- TMR hoạt động dựa trên nguyên tắc ba lần lặp: ba mô-đun chức năng giống hệt nhau thực hiện cùng một tác vụ và kết quả của chúng được đưa vào một bộ lọc hậu xử lý, thường là một bộ biểu quyết, để chọn ra kết quả cuối cùng. Nếu một trong ba mô-đun gặp sự cố, hai mô-đun còn lại đảm bảo rằng hệ thống vẫn hoạt động chính xác, qua đó giảm thiểu rủi ro hệ thống thất bại do lỗi đơn lẻ.

Kỹ thuật cải tiến độ tin cậy



Độ tin cậy TMR so sánh với độ tin cậy Simplex

Kỹ thuật cải tiến độ tin cậy

Theo thời gian tổng số lỗi phát sinh ra khi áp dụng kỹ TMR cao hơn mô hình đơn giản, nhưng ngược lại hệ thống vẫn tiếp tục hoạt động vì có mô-đun dự phòng có thể sửa khắc phục lỗi khi xảy ra sự cố như đã đề cập ở trên. Lỗi xuất hiện theo TMR cao hơn mô hình simplex có thể ước tính dựa vào công thức sau:

$$t = T \times 2$$

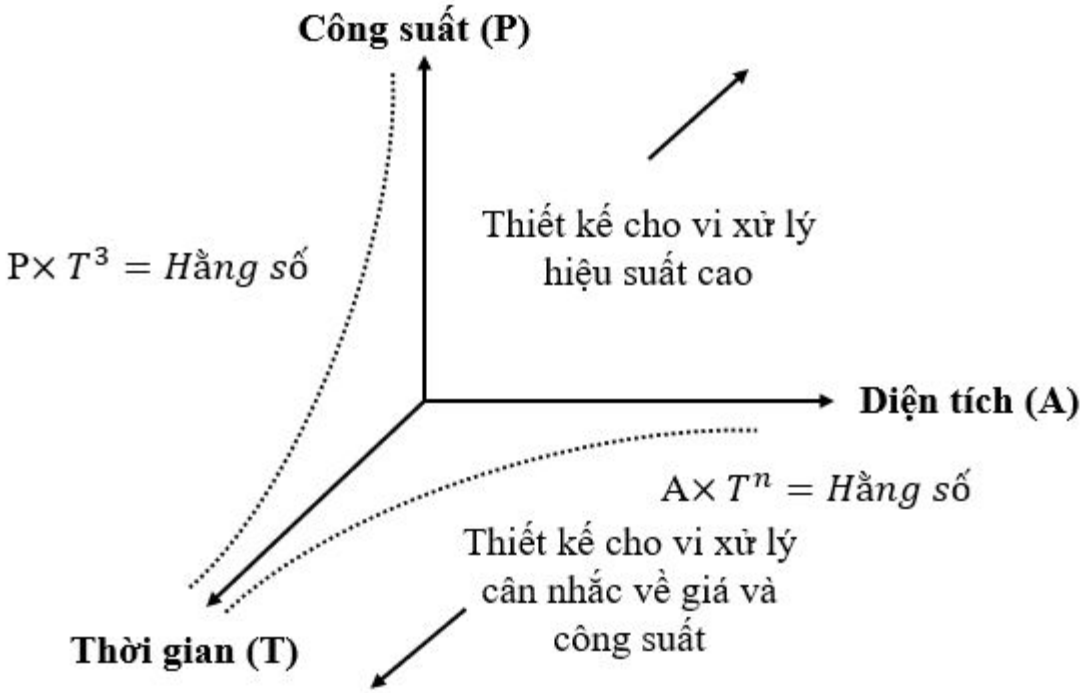
Khắc phục lỗi từ nhà máy sản xuất chip

Thiết kế với khả năng kiểm thử (design for testability - DFT):

- Kỹ thuật scan cho phép cấu hình dữ liệu được xác định trước và lưu trữ ở đầu vào, đầu ra của các san được so sánh với các kết quả đã xác định trước. Chuỗi scan trong hình thức đơn giản nhất của nó bao gồm một điểm nhập và một điểm xuất riêng biệt từ mỗi ô nhớ. Mỗi điểm này được ghép nối (multiplexed) vào một bus nối tiếp, có thể được nạp vào/ra từ ô nhớ độc lập với phần còn lại của hệ thống.
- Sự tích hợp của các chuỗi scan vào thiết kế soc là một phần quan trọng trong quy trình phát triển, giúp giảm thiểu rủi ro và đảm bảo rằng sản phẩm cuối cùng sẽ hoạt động đúng như mong đợi khi được triển khai trên thực tế.

Sự đánh đổi trong hệ thống SoC

Trong các thiết kế bộ xử lý, tồn tại một giới hạn được biểu diễn bởi phương trình **A × T^n**, trong đó n thường có giá trị từ 1 đến 2, phản ánh sự đánh đổi giữa **diện tích A** và **thời gian thực thi T**. Ngoài ra, cũng tồn tại một **mối quan hệ đánh đổi giữa thời gian T và công suất P, biểu diễn qua giới hạn P ×T^3**.



Minh họa sự đánh đổi trong thiết kế vi xử lý

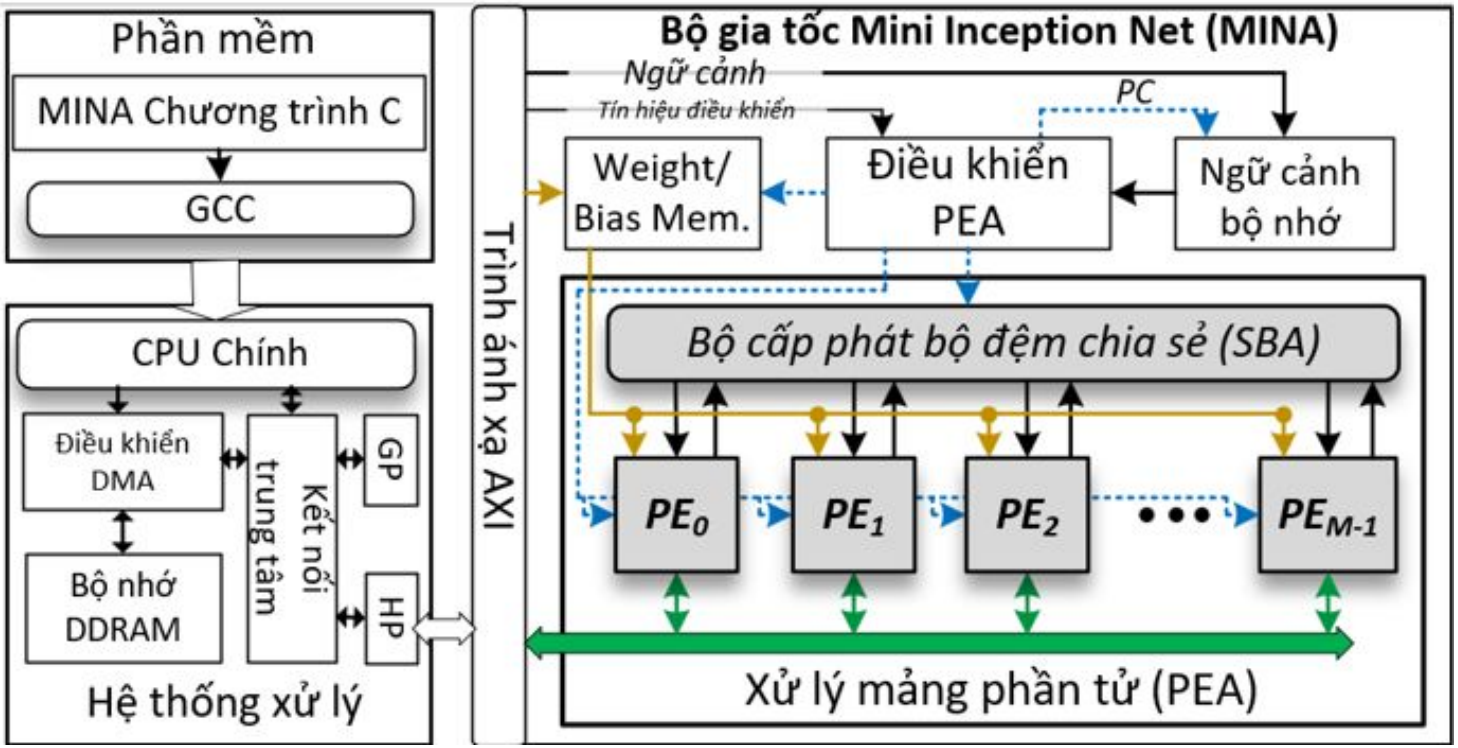
Tích hợp hệ thống

- Tính tích hợp hệ thống là một trong những ưu điểm nổi bật của SoC trên FPGA, cho phép gói gọn nhiều chức năng phức tạp trên một chip duy nhất.
- SoC trên FPGA gồm các thành phần như CPU, bộ nhớ, giao thức bus, và các khối IP được tích hợp chặt chẽ trên cùng một nền tảng.

⇒ Giảm kích thước hệ thống, cải thiện hiệu năng, giảm độ trễ truyền dữ liệu, và tăng cường khả năng tương tác giữa các khối chức năng.

- **Phần cứng** bao gồm các **IP cứng** như CPU, AXI bus, bộ điều khiển DMA, và bộ nhớ DDRAM.
- **Phần mềm** của hệ thống, được viết bằng ngôn ngữ C và biên dịch bằng GCC, tạo ra các cấu hình (CTX) để định nghĩa tham số mạng nơ-ron tích chập và điều khiển phần cứng.

Tích hợp hệ thống



Minh họa một hệ thống SoC thiết kế kiến trúc mạng Inception Net

Câu hỏi và bài tập

1. Một SoC điều khiển động cơ bao gồm CPU hoạt động ở tần số 500 MHz. Một bộ nhớ ngoài có độ trễ truy cập là 10 chu kỳ CPU. Bus I2C truyền dữ liệu với độ trễ mỗi chiều là 50 chu kỳ. Một khối IP điều khiển động cơ hoạt động ở 100 MHz, cần 1,000 chu kỳ để xử lý một lệnh điều khiển. Hệ thống hỗ trợ 8 tầng pipeline. Tính tổng độ trễ của hệ thống khi xử lý một lệnh điều khiển động cơ trong hai trường hợp:
   a) Không sử dụng pipeline.
   b) Có sử dụng pipeline.

3. Giả sử bạn đang thiết kế một hệ thống SoC trên FPGA với hai thiết kế khác nhau cho cùng ứng dụng. Thiết kế 1 sử dụng 5000 LUTs, 2000 FFs, 20 DSP (1 DSP bằng 280 LUT và 32 FF), 10 BRAM (1 BRAM bằng 1,600 LUT và 32 FF). Thiết kế 2 sử dụng 6000 LUTs, 2500 FFs, 10 DSP (1 DSP bằng 280 LUT và 32 FF), 7 BRAM (1 BRAM bằng 1,600 LUT và 32 FF) . Xác định thiết kế nào tối ưu hơn

Câu hỏi và bài tập

1. Giả sử bạn đang thiết kế một hệ thống SoC trên FPGA với hai thiết kế khác nhau cho cùng ứng dụng. Thiết kế 1 sử dụng 5000 LUTs, 2000 FFs, 20 DSP (1 DSP bằng 280 LUT và 32 FF), 10 BRAM (1 BRAM bằng 1,600 LUT và 32 FF) và có công suất tiêu thụ là 3 W với độ trễ 500 mili giây. Thiết kế 2 sử dụng 6000 LUTs, 2500 FFs, 10 DSP (1 DSP bằng 280 LUT và 32 FF), 7 BRAM (1 BRAM bằng 1,600 LUT và 32 FF) và có công suất tiêu thụ là 2.5 W với độ trễ 200 mili giây. Xác định thiết kế nào hiệu quả hơn.

2. Một hệ thống SoC có CPU, bộ nhớ và một IP tự thiết kế thực hiện bài toán tính tổng của dãy số có NNN phần tử. Mỗi phần tử là một số thực (float 32-bit). Tần số hoạt động của FPGA là 250 MHz, tần số hoạt động của IP tự thiết kế là 500 MHz. Độ trễ truy cập bộ nhớ là 80 ns, độ trễ bus là 40 ns, các độ trễ khác xem như bỏ qua. Biết mỗi phép tính cộng tốn 2 ns, và mỗi phép tính cần tải dữ liệu từ bộ nhớ một lần. Hãy tính toán:
   a) Tổng thời gian thực hiện và thông lượng của hệ thống trong trường hợp IP thiết kế không xử lý song song.
   b) Tổng thời gian thực hiện và thông lượng của hệ thống trong trường hợp IP thiết kế có xử lý song song với 16 nhân xử lý.

- Hiện thực hệ thống SoC cho bài toán nhập vào 1 số N và tính tổng giá trị của nó từ 0 đến N. Với vỏ bọc AXI_lite, thực hiện tính hardware resource, power, fmax của hệ thống.